

EIN092B: Laboratorio de Técnicas de Visualización y Reducción de Dimensionalidad

4 de septiembre de 2026

1. Objetivos

- Explorar conjuntos de datos usando técnicas de visualización y reducción de la dimensionalidad.
- Entender los conceptos asociados a las técnicas PCA, t-SNE y UMAP, resaltando las ventajas y desventajas de cada una.

2. Indicaciones generales.

Las actividades planteadas en esta guía deben ser entregadas durante la sesión de laboratorio.

2.1. Sobre el proceso de revisión de actividades.

Para el proceso de revisión consideren lo siguiente:

- La revisión y evaluación de cada una de las actividades se realizará en dos etapas: (i) una validación básica de requerimientos funcionales mínimos durante la sesión, y (ii) una verificación de requerimientos funcionales y no funcionales posterior a la sesión.
- Durante la sesión, avise al staff (profesor o ayudante) cuando complete una o varias actividades para que muestre y explique sus resultados, y responda las preguntas que se le realizarán sobre los conceptos asociados.
- Cualquier integrante del grupo debe ser capaz de responder las preguntas. Si las respuestas no son satisfactorias, o algún integrante no es capaz de responder, no se revisará la actividad hasta que demuestren que TODOS los integrantes del grupo demuestren un entendimiento mínimo sobre lo que están entregando.
- Durante la sesión, el staff marcará en una planilla cuando Ud. considere una actividad terminada y haya pasado la interrogación. El tener la actividad marcada como revisada en la sesión no entrega puntaje, y solo asegura que cumple con requerimientos para que entregue sus archivos y pase a la siguiente etapa de revisión. El formato de entrega de sus archivos se indica en la Sección 2.2.

- En el caso de las actividades no entregadas en la sección correspondiente, contará como no entregada a tiempo y **deberá entregar la actividad corregida en la siguiente sesión, aplicando descuentos por atraso (penalización correspondiente al 35 % del puntaje inicial si se entrega en la siguiente sesión. Si se entrega en dos sesiones posteriores, 50 %).**
- En el caso de que algún integrante del grupo no este presente cuando su grupo correspondiente presente la evaluación y su correspondiente interrogación, deberá presentarlo por su propia cuenta en las siguientes sesiones y sufriendo la penalización por atraso.

2.2. Indicaciones de formato para entrega.

La evaluación constará de dos partes. La primera parte consiste en una validación básica y preguntas sobre los códigos propuestos por el grupo. La segunda parte constará de una verificación de sus códigos y respuestas, la cual se realizará sobre los archivos que deberá entregar vía Aula USM, los cuales deben seguir estrictamente las especificaciones que se le indican a continuación:

1. Debe crear una carpeta con el nombre *GXX_Sesion4*, donde *XX* el número de grupo. Por ejemplo, la carpeta para el grupo 5 se llamaría *G5_Sesion4*. El número de su grupo fue definido en la sesión anterior.
2. Dentro de esta carpeta, debe incluirse un archivo de Jupyter Notebook que contenga el desarrollo de las actividades y las respuestas correspondientes.

Recordar que **será responsabilidad del grupo entregar de manera correcta los archivos**. Se permitirá el envío de los archivos hasta las **23:59 del 4 de Septiembre**. Sin embargo, solo se revisarán las actividades marcadas como revisadas durante la sesión. Cualquier actividad adicional no será considerada en la revisión.

3. Introducción

En cursos anteriores, se ha hecho énfasis principalmente en el uso de métodos de aprendizaje supervisado para resolver problemas de regresión y clasificación. Sin embargo, en gran parte de los problemas no se conoce una variable de respuesta ni una etiqueta para un conjunto de características o mediciones. Bajo este escenario, dado un conjunto de características $X_1, X_2, \dots, X_p$ adquiridas o medidas en n observaciones, el objetivo es encontrar tendencias que proporcionen información sobre el conjunto de mediciones. ¿Qué técnicas permiten visualizar los datos resaltando las características o descubriendo subgrupos dentro de las características o de las mediciones? El aprendizaje no supervisado abarca un conjunto diverso de técnicas para resolver interrogantes como este, ya que suele emplearse para realizar análisis exploratorio de datos.

En esta guía de laboratorio nos enfocaremos en técnicas de *reducción de dimensionalidad y visualización*: (i) *análisis de componentes principales* (PCA), una herramienta utilizada para la visualización de datos o para el preprocesamiento de datos antes de aplicar técnicas de aprendizaje supervisado; (ii) t -SNE, una técnica no lineal basada en probabilidades para visualización de datos de alta dimensionalidad [1]; y (iii) *Uniform Manifold Approximation and Projection* (UMAP), otra técnica no lineal basada en teoría de variedades (*manifold learning*) y en la construcción de un grafo

de vecindades, que suele ser más rápida que t-SNE y tiende a preservar mejor la estructura global de los datos [3].

4. Actividades

En las siguientes subsecciones se describen las actividades a realizar. En cada actividad encontrará indicaciones generales sobre el conjunto de datos a utilizar e indicaciones generales. En este laboratorio, se emplearán técnicas de visualización de datos de alta dimensionalidad para el análisis exploratorio de datos.

4.1. PCA (30 Puntos)

PCA es una herramienta fundamental para el análisis de datos en alta dimensión. Se define formalmente como una transformación lineal ortogonal que mapea los datos a un nuevo sistema de coordenadas, de tal manera que la mayor varianza de cualquier proyección en los datos se encuentra en la primera componente, la segunda mayor varianza en la segunda componente, y así sucesivamente. PCA se clasifica como una técnica de aprendizaje no supervisado, y es comúnmente utilizada para la compresión de datos sin pérdidas significativas¹, la eliminación de redundancias en imágenes y la selección de características, lo que resulta útil en distintos campos.

Existen múltiples enfoques para mapear los datos a un nuevo sistema de coordenadas. El enfoque más común consiste en calcular los vectores y valores propios de la matriz de covarianza, la cual se computa a partir de las variables de entrada. Sin embargo, también existen otros métodos, como la *descomposición en valores singulares* (SVD) [2]. La elección del método para realizar el mapeo comúnmente depende de la capacidad de cómputo disponible y el tamaño de las matrices. En la mayoría de los casos, los usuarios se abstraen de los detalles de bajo nivel que esconden las bibliotecas y pueden realizar la transformación sin necesidad de comprender todos los detalles (simplifica el trabajo cuando se conoce el funcionamiento de la técnica).

La primera actividad se enfoca en el uso de PCA, empleando el *USArrests* dataset. Este dataset contiene el número de detenciones por cada 100.000 residentes para cada uno de los estados de Estados Unidos, y se especifica el número de detenciones para los delitos de agresión, asesinato y violación. También se registra la variable *UrbanPop* que corresponde al porcentaje de la población de cada estado que vive en zonas urbanas.

¹Un ejemplo práctico de esto es el método de *Eigenfaces* (ejemplo de clase), que se utiliza en el reconocimiento facial.

Actividades a realizar:

1. Cargue el conjunto de datos en un dataframe. Extraiga la etiqueta de los datos y las imágenes correspondientes.

```
1 import pandas as pd
2
3 data = pd.read_csv(PATH.DATASET)
4 y_train = data.iloc[:, 0].values
5 x_train = data.iloc[:, 1:].values
6
7 print("Dimensiones de x_train:", x_train.shape)
8 print("Dimensiones de y_train:", y_train.shape)
```

Seguidamente, estandarizar cada variable para que tenga media cero y desviación estándar uno. Para un conjunto de datos $\mathbf{X}$, se estandarizan los datos de la siguiente forma:

$$\mathbf{X} = \frac{\mathbf{X} - \mu\mathbf{X}}{\sigma\mathbf{X}}, \quad (1)$$

donde $\mu\mathbf{X}$ y $\sigma\mathbf{X}$ corresponden a la media y la desviación estándar de $\mathbf{X}$, respectivamente.

2. Transforme los datos con la técnica PCA (componentes $n = 2$). Reporte la varianza contenida en las dos primeras componentes.
3. Realice un gráfico de dispersión usando las dos primeras componentes. Además, sobre este gráfico añada los vectores propios; para esto se requiere dibujar un vector desde el origen del plano hasta las coordenadas en los vectores propios. Estas coordenadas indican las direcciones de máxima varianza y muestran cómo las variables originales (Murder, Assault, UrbanPop y Rape) contribuyen a cada componente principal.

Las funciones `pca.explained_variance_` y `pca.components_` permiten obtener los valores y vectores propios, respectivamente. A continuación, se proporciona un código de referencia para hacer una gráfica de los vectores propios de la transformación e incluir el nombre de cada variable. De forma similar, para el gráfico de dispersión de los datos transformados, incluya el nombre de cada estado sobre las coordenadas en el plano cartesiano.

```
1 eigenvectors = pca.components_
2 for i in range(4):
3     plt.quiver(0, 0, eigenvectors[0, i], eigenvectors[1, i],
4               angles='xy', scale_units='xy', scale=1, color='r')
5
6 plt.title('PCA: Datos transformados y Eigenvectores')
7
8 for i, word in enumerate(['Murder', 'Assault', 'UrbanPop', 'Rape']):
9     plt.annotate(word, (eigenvectors[0, i], eigenvectors[1, i]))
```

4. Discuta sobre qué variables tienen mayor impacto en las direcciones de mayor varianza en los datos. ¿Por qué crees que Murder, Assault y Rape están ubicadas cerca unas de otras en el espacio de los componentes principales? ¿Qué indica esto sobre la relación entre estas variables? ¿Qué podría significar el hecho de que UrbanPop esté más lejos de las otras tres variables?

4.2. Visualización de datos usando t-SNE (40 puntos)

La visualización de datos en alta dimensionalidad es un problema importante en muchos dominios y se ocupa para visualizar conjuntos de datos de dimensionalidad muy variada. En esta sección, nos enfocaremos en una técnica llamada t-SNE, una herramienta que suele emplearse para la visualización de datos de similitud, la cual es capaz de retener la estructura local de los datos mientras revela alguna estructura global de los datos. t-SNE permite obtener una intuición sobre cómo se organizan los datos en dimensiones superiores. Comúnmente, se usa para visualizar conjuntos de datos complejos en dos y tres dimensiones, lo que nos permite comprender sobre los patrones subyacentes y las relaciones existentes en los datos. Por lo tanto, se podría categorizar como una técnica para explorar un determinado conjunto de datos a fin de encontrar relaciones subyacentes, sin requerir el conocimiento previo de la clase o categoría de cada observación.

t-SNE es definida por sus autores como una técnica para crear un mapa que muestre la estructura de los datos a múltiples escalas diferentes. Particularmente, esto es importante para conjuntos de alta dimensión, como imágenes de objetos de múltiples clases vistos desde múltiples puntos de vista. Generalmente, se emplea el dataset MNIST para realizar evaluaciones preliminares; sin embargo, en esa sección emplearemos el dataset *Olivetti faces*.



(a) Imágenes extraídas arbitrariamente del conjunto de datos.



(b) Imágenes centradas en la media.

Figura 1: Ejemplos del conjunto de datos de *Olivetti faces*.

Como primer paso, cargue el dataset. El conjunto de datos *Olivetti faces* consta de imágenes de 40 individuos con pequeñas variaciones en el punto de vista, grandes variaciones en la expresión y la adición ocasional de gafas. Este conjunto de datos contiene un conjunto de imágenes de caras tomadas entre abril de 1992 y abril de 1994 en *AT&T Laboratories Cambridge*. El conjunto de datos consta de 400 imágenes (10 por individuo) de tamaño $92 \times 112 = 10.304$ píxeles, y está etiquetado según la identidad. En la Figura 1 se muestran algunos ejemplos de las imágenes que conforman el conjunto de datos.

A continuación, se proporciona un fragmento de código para cargar el dataset ²:

```

1 import logging
2
3 import matplotlib.pyplot as plt
4 from numpy.random import RandomState
5
6 from sklearn import cluster, decomposition
7 from sklearn.datasets import fetch_olivetti_faces
8
9 rng = RandomState(0)
10
11 # Display progress logs on stdout
12 logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(
13     message)s")
14
15 faces, _ = fetch_olivetti_faces(return_X_y=True, shuffle=True, random_state=rng)
16 n_samples, n_features = faces.shape
17
18 # Global centering (focus on one feature, centering all samples)
19 faces_centered = faces - faces.mean(axis=0)
20
21 # Local centering (focus on one sample, centering all features)
22 faces_centered -= faces_centered.mean(axis=1).reshape(n_samples, -1)
23
24 print("Dataset consists of %d faces" % n_samples)

```

Actividades a realizar para los experimentos de esta sección:

1. Obtenga una nueva representación del conjunto de datos utilizando PCA para reducir la dimensionalidad de los datos a 30. Seguidamente, utiliza el algoritmo t-SNE para convertir la representación de 30 dimensiones en un mapa bidimensional. Haga un gráfico con la varianza contenida en las 30 primeras componentes principales obtenidas.
2. Muestre la transformación obtenida en un gráfico de dispersión y asegúrese de mostrar la información de cada clase usando un color y/o símbolo a fin de poder evaluar visualmente si el gráfico de dispersión muestra alguna relación o similitudes dentro de cada clase. En este caso, para una configuración del parámetro de `perplexity = 40`, inspeccione un subgrupo de observaciones en el gráfico de dispersión y compare las imágenes originales ¿Encuentra alguna relación entre la similitud entre la transformación obtenida con t-SNE?
3. Haga uso del *widget* interactivo de IPython el cual reciba el parámetro de `perplexity` y permita visualizar la transformación obtenida con t-SNE usando distintos valores de este parámetro.

```

1 from ipywidgets import interactive
2 from sklearn.manifold import TSNE
3
4 def tsne_pipeline(param):
5     ...
6     # define your pipeline
7     tsne = TSNE(n_components=2, verbose=1, perplexity= 30, random_state=0)

```

²Extraído del enlace: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.fetch_olivetti_faces.html

```

8     z = tsne.fit_transform(x)
9
10    p_min = -10 #define your range
11    p_max = 10 #define your range
12    interactive(tsne_pipeline, param=(p_min, p_max))

```

- Discuta sobre la selección de este parámetro y su importancia en la transformación obtenida.
 - Describa que sucede al aumentar o disminuir el parámetro `perplexity`. Comente: ¿Qué valor de `perplexity` elegiría en este caso?
4. Compare visualmente la representación obtenida con t-SNE (a partir de representación de PCA de 30 componentes) con la representación obtenida usando PCA (con $n = 2$) para ver cuál proporciona mejor separación entre clústeres.

4.3. Reducción de dimensionalidad con UMAP (30 puntos)

Uniform Manifold Approximation and Projection (UMAP) es una técnica de reducción de dimensionalidad no lineal basada en *manifold learning*³ y en fundamentos de topología [3]. A diferencia de PCA, UMAP no busca preservar la varianza global mediante una transformación lineal; en cambio, construye un grafo ponderado de vecindades en el espacio de alta dimensión y optimiza una representación de baja dimensión cuya estructura topológica sea lo más similar posible a la del grafo original. Este enfoque le permite a UMAP preservar tanto relaciones locales como, en mayor medida que t-SNE, la estructura global de los datos, además de ser habitualmente más rápido en tiempo de cómputo.

Los hiperparámetros más relevantes de UMAP son:

- `n_neighbors`: Controla el tamaño del vecindario local que UMAP utiliza para aproximar la estructura de la variedad. Valores pequeños priorizan la estructura local (similar a una `perplexity` baja en t-SNE), mientras que valores grandes favorecen una visión más global de los datos.
- `min_dist`: Controla qué tan compactos pueden estar los puntos en la representación de baja dimensión. Valores pequeños generan agrupaciones más compactas y densas, mientras que valores grandes producen una distribución más dispersa que preserva mejor la topología global.
- `metric`: Define la métrica de distancia utilizada para calcular las similitudes entre observaciones en el espacio de alta dimensión. La elección de la métrica debe ajustarse a la naturaleza de los datos, considerando factores como la dimensionalidad, la dispersión, o la información contenida en las distancias..

Para esta sección, reutilizaremos el conjunto de datos *Fashion-MNIST*, conformado por 60.000 imágenes de artículos de vestuario. Cada imagen contenida en el dataset está representada con una profundidad de pixel de 28×28 y asociada a una etiqueta de entre 10 clases disponibles.

³Manifold learning es una familia de técnicas de reducción de dimensionalidad que asume que los datos de alta dimensión están relacionados con una variedad (manifold) de dimensión más baja, embebida en ese espacio de alta dimensión. Los métodos de manifold learning intentan preservar esa estructura no lineal, típicamente basándose en relaciones de vecindad local.

A continuación, se proporciona un fragmento de código de referencia para cargar el dataset y aplicar UMAP:

```
1 import umap
2 import numpy as np
3 from sklearn.datasets import fetch_openml
4
5 fashion_mnist = fetch_openml('Fashion-MNIST', version=1, as_frame=False)
6 x_train = fashion_mnist.data.astype(np.float32) / 255.0
7 y_train = fashion_mnist.target.astype(int)
8
9 print("Dimensiones de x_train:", x_train.shape)
10 print("Dimensiones de y_train:", y_train.shape)
11
12 reducer = umap.UMAP(n_neighbors=15, min_dist=0.1, n_components=2, random_state=0)
13 embedding = reducer.fit_transform(x_train)
```

Actividades a realizar en esta subsección:

1. Instale la librería `umap-learn`. Tal como se hizo con t-SNE, reduzca primero la dimensionalidad de los datos originales de Fashion-MNIST a 30 componentes mediante PCA, y aplique UMAP sobre esta representación para obtener una proyección en 2 dimensiones. Grafique la transformación obtenida, coloreando cada punto de acuerdo a su clase.
2. Realice un benchmarking variando los parámetros `n_neighbors` y `min_dist`, utilizando valores pequeños, intermedios y grandes para cada uno, y grafique las transformaciones obtenidas. Discuta cómo la variación de `n_neighbors` afecta la preservación de estructuras locales y globales, y cómo `min_dist` modifica la compacidad de los grupos visualizados. A partir de los resultados, indique qué configuración considera más adecuada para representar las imágenes de Fashion-MNIST y justifique su elección.
3. Realice un benchmarking variando el parámetro `metric`, manteniendo fijos `n_neighbors` y `min_dist`. Investigue brevemente cómo se calcula cada una de estas distancias y discuta qué diferencias observa en las transformaciones obtenidas. ¿Qué variante le parece más adecuada para representar imágenes como las de Fashion-MNIST? ¿En qué otros tipos de datos esperaría que otras métricas sean más apropiadas, y por qué?
4. Haga uso de un *widget* interactivo de IPython que reciba los parámetros `n_neighbors` y `min_dist`, y permita visualizar la transformación obtenida con UMAP usando distintas combinaciones de estos parámetros.

```
1 from ipywidgets import interactive
2 import umap
3
4 def umap_pipeline(n_neighbors, min_dist):
5     ...
6     # define your pipeline
7     reducer = umap.UMAP(n_neighbors=n_neighbors, min_dist=min_dist,
8                         n_components=2, random_state=0)
9     z = reducer.fit_transform(x_train)
10
11 interactive(umap_pipeline, n_neighbors=(2, 200), min_dist=(0.0, 1.0))
```


5. UMAP suele compararse directamente con t-SNE, ya que ambas son técnicas no lineales orientadas a la visualización de datos de alta dimensionalidad, y ambas se benefician de reducir primero el ruido y el costo computacional mediante un preprocesamiento con PCA. Compare entonces sobre el mismo conjunto de datos, usando en ambos casos la representación de 30 componentes de PCA como entrada y fijando los hiperparámetros en un valor razonable único, la transformación obtenida con t-SNE y la obtenida con UMAP. ¿Qué técnica logra una mejor separación visual entre clases? ¿En qué escenarios preferiría usar UMAP por sobre t-SNE, o viceversa?
6. UMAP admite, de manera opcional, el uso de las etiquetas de clase durante el ajuste del modelo (`reducer.fit(x_train_pca, y=y_train)`), lo que da lugar a una proyección *semi-supervisada* que incorpora esta información en la construcción del grafo de vecindades. Obtenga esta representación semi-supervisada, utilizando igualmente la representación de 30 componentes de PCA como entrada, y compárela visualmente con la versión no supervisada obtenida en el primer punto de esta subsección. ¿Qué cambios observa en la separación entre clases? ¿En qué escenarios reales tendría sentido (o no) utilizar esta variante semi-supervisada?

Referencias

- [1] Laurens Van der Maaten y Geoffrey Hinton. “Visualizing data using t-SNE.” En: *Journal of machine learning research* 9.11 (2008).
- [2] Zhihua Zhang. “The singular value decomposition, applications and beyond”. En: *arXiv preprint arXiv:1510.08532* (2015).
- [3] Leland McInnes, John Healy y James Melville. *UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction*. 2020. arXiv: 1802.03426 [stat.ML]. URL: <https://arxiv.org/abs/1802.03426>.